

Document number	P3230R0
Date	2024-04-05
Audience	LEWG, SG9 (Ranges)
Reply-to	Hewill Kang <hewillk@gmail.com>

views::(take|drop)_exactly

- - Abstract
 - Revision history
 - Discussion
 - Design
 - Implementation experience
 - Proposed change
 - References

Abstract

This paper proposes Tier 1 adaptors in [P2760](#): `views::drop_exactly` and `views::take_exactly`, which are cousins of `drop` and `take`, to improve the C++26 ranges facilities.

Revision history

R0

Initial revision.

Discussion

`drop_exactly` and `take_exactly` are very similar to `drop` and `take`, the only difference is that the former requires that `N` must not be greater than the number of elements in the original range; otherwise, it will be NB, which is exactly the "exactly" part.

Since there is no bounds check, `views::meow_exactly` is more efficient than `views::meow` for situations where the user already knows that there are enough elements in the range.

Design

Should we introduce `drop_exactly_view`/`take_exactly_view` classes?

Although both can achieve similar effects with existing utilities, e.g. `take_exactly(r, N)` is somewhat equivalent to `views::counted(ranges::begin(r), N)`, and `drop_exactly` is somewhat equivalent to `subrange(ranges::next(std::ranges::begin(r), N), ranges::end(r))`, these are not fully replaceable due to certain limitations. The main issue is that both need to apply `ranges::begin` to extract the iterator of the origin range, which may be ill-formed when taking rvalue ranges, not to mention that `drop_exactly_view` also needs to cache the result to meet the time complexity requiring by the `range` concept.

Since we've already assumed that the range is large enough, out-of-bounds cases are ignored in the implementation, which means that `meow_exactly_view` is just a reduced version of `meow_view`. So introducing these classes doesn't bring much complexity to the standard.

Should we specialized for return types?

Currently, both `take` and `drop` have optimized the return type, that is, when it takes an object of range type `empty_view`, `span`, `string_view`, `subrange`, `iota_view` and `repeat_view`, it will return an object of the same type to reduce template instantiation. There is no reason not to apply similar optimizations to `take_exactly` and `drop_exactly`.

However, it should be noted that compared to the former, since we no longer care whether the specialized type models `sized_range`, `views::meow_exactly` applied to the infinite range may downgrade it to a finite range. For example, `views::iota(0) | views::take_exactly(5)` will produce `views::iota(0, 5)`, and `views::iota(0) | views::drop_exactly(5)` will produce `views::iota(5)`, this applies to `subrange<int*, unreachable_sentinel_t>` as well.

Other *exactly* family?

Although [P2760](#) only mentions `take_exactly` and `drop_exactly`, it is natural to think whether we need `slice_exactly`, `take_last_exactly` and `drop_last_exactly`, etc. Considering the bandwidth of LEWG, this paper does not intend to introduce these potentially useful utilities.

Implementation experience

The author implemented `views::(take|drop)_exactly` based on libstdc++, see [here](#).

Proposed change

This wording is relative to [N4971](#).

1. Add a new feature-test macro to 17.3.2 [\[version.syn\]](#):

```
#define __cpp_lib_ranges_take_exactly_2024XXL // freestanding, also in <ranges>
#define __cpp_lib_ranges_drop_exactly_2024XXL // freestanding, also in <ranges>
```

2. Modify 26.2 [\[ranges.syn\]](#), Header `<ranges>` synopsis, as indicated:

```
#include <compare>           // see \[compare.syn\]
#include <initializer_list>   // see \[initializer.list.syn\]
#include <iterator>           // see \[iterator.synopsis\]

namespace std::ranges {
    [...]
    namespace views { inline constexpr unspecified take = unspecified; }           // freestanding

    // \[range.take.exactly\], take exactly view
    template<view> class take exactly view;                                       // freestanding

    template<class T>
        constexpr bool enable borrowed range<take exactly view<T>> =           // freestanding
            enable borrowed range<T>;

    namespace views { inline constexpr unspecified take exactly = unspecified; }   // freestanding
    [...]
    namespace views { inline constexpr unspecified drop = unspecified; }          // freestanding

    // \[range.drop.exactly\], drop exactly view
    template<view V> class drop exactly view;                                    // freestanding

    template<class T>
        constexpr bool enable borrowed range<drop exactly view<T>> =           // freestanding
            enable borrowed range<T>;

    namespace views { inline constexpr unspecified drop exactly = unspecified; }   // freestanding
    [...]
}
```

3. Add 26.7.? **Take exactly view** [\[range.take.exactly\]](#) after 26.7.10 [\[range.take\]](#) as indicated:

[26.7.?1] Overview [\[range.take.exactly.overview\]](#)

-1- `take_exactly_view` produces a view of the first *N* elements from another view. The behavior is undefined if the adapted view contains fewer than *N* elements.

-2- The name `views::take_exactly` denotes a range adaptor object (26.7.2 [\[range.adaptor.object\]](#)). Let *E* and *F* be expressions, let *T* be `remove_cvref_t<decltype((E))>`, and let *D* be `range_difference_t<decltype((E))>`. If `decltype((F))` does not model `convertible_to<D>`, `views::take_exactly(E, F)` is ill-formed. Otherwise, the expression `views::take_exactly(E, F)` is expression-equivalent to:

(2.1) — if *T* is a specialization of `empty_view` (26.6.2.2 [\[range.empty.view\]](#)), then `((void)F, decay-copy(E))`, except that the evaluations of *E* and *F* are indeterminately sequenced.

(2.2) — Otherwise, if *T* models `random_access_range` and is a specialization of `span` (24.7.2.2 [\[views.span\]](#)), `basic_string_view` (23.3 [\[string.view\]](#)), or `ranges::subrange` (26.5.4 [\[range.subrange\]](#)), then

`U(ranges::begin(E), ranges::begin(E) + static_cast<D>(F))`, except that `E` is evaluated only once, where `U` is a type determined as follows:

(2.2.1) — if `T` is a specialization of `span`, then `U` is `span<typename T::element_type>`;

(2.2.2) — otherwise, if `T` is a specialization of `basic_string_view`, then `U` is `T`;

(2.2.3) — otherwise, `T` is a specialization of `subrange`, and `U` is `subrange<iterator_t<T>>`;

(2.3) — otherwise, if `T` is a specialization of `iota_view` (26.6.4.2 [\[range.iota.view\]](#)) that models `random_access_range`, then `iota_view(*ranges::begin(E), *(ranges::begin(E) + static_cast<D>(F)))`, except that `E` is evaluated only once.

(2.4) — Otherwise, if `T` is a specialization of `repeat_view` (26.6.5.2 [\[range.repeat.view\]](#)), then `views::repeat(*E.value_, static_cast<D>(F))`.

(2.5) — Otherwise, `take_exactly_view(E, F)`.

-3- *[Example 1:*

```
auto ints = views::iota(0) | views::take_exactly(10);
for (auto i : ints | views::reverse) {
    cout << i << ' '; // prints 9 8 7 6 5 4 3 2 1 0
}
```

— *end example]*

[26.7.?.2] Class template `take_exactly_view` [\[range.take.exactly.view\]](#)

```
namespace std::ranges {
    template<view V>
    class take_exactly_view : public view_interface<take_exactly_view<V>> {
    private:
        V base_ = V(); // exposition only
        range_difference_t<V> count_ = 0; // exposition only

    public:
        take_exactly_view() requires default_initializable<V> = default;
        constexpr explicit take_exactly_view(V base, range_difference_t<V> count);

        constexpr V base() const & requires copy_constructible<V> { return base_; }
        constexpr V base() && { return std::move(base_); }

        constexpr auto begin() requires (!simple-view<V>) {
            if constexpr (random_access_range<V>)
                return ranges::begin(base_);
            else
                return counted_iterator(ranges::begin(base_), count_);
        }

        constexpr auto begin() const requires range<const V> {
            if constexpr (random_access_range<const V>)
                return ranges::begin(base_);
            else
                return counted_iterator(ranges::begin(base_), count_);
        }

        constexpr auto end() requires (!simple-view<V>) {
            if constexpr (random_access_range<V>)
                return ranges::begin(base_) + count_;
            else
                return default_sentinel;
        }

        constexpr auto end() const requires range<const V> {
            if constexpr (random_access_range<const V>)
                return ranges::begin(base_) + range_difference_t<const V>(count_);
            else
                return default_sentinel;
        }

        constexpr auto size() const noexcept { return to_unsigned_like(count_); }
    };

    template<class R>
    take_exactly_view(R&&, range_difference_t<R>)
}
```

```

    }
    -> take_exactly_view<views::all_t<R>>;
constexpr explicit take_exactly_view(V base, range_difference_t<V> count);

```

-1- *Preconditions*: `count <= ranges::distance(base)` is true.

-2- *Effects*: Initializes `base_` with `std::move(base)` and `count_` with `count`.

4. Add **26.7.2 Drop exactly view** [`range.drop.exactly`] after 26.7.12 [`range.drop`] as indicated:

[26.7.2.1] Overview [`range.drop.exactly.overview`]

-1- `drop_exactly_view` produces a view excluding the first *N* elements from another view. The behavior is undefined if the adapted view contains fewer than *N* elements.

-2- The name `views::drop_exactly` denotes a range adaptor object (26.7.2 [`range.adaptor.object`]). Let *E* and *F* be expressions, let *T* be `remove_cvref_t<decltype((E))>`, and let *D* be `range_difference_t<decltype((E))>`. If `decltype((F))` does not model `convertible_to<D>`, `views::drop_exactly(E, F)` is ill-formed. Otherwise, the expression `views::drop_exactly(E, F)` is expression-equivalent to:

(2.1) — if *T* is a specialization of `empty_view` (26.6.2.2 [`range.empty.view`]), then `((void)F, decay-copy(E))`, except that the evaluations of *E* and *F* are indeterminately sequenced.

(2.2) — Otherwise, if *T* models `random_access_range` and is

(2.2.1) — a specialization of `span` (24.7.2.2 [`views.span`]),

(2.2.2) — a specialization of `basic_string_view` (23.3 [`string.view`]),

(2.2.3) — a specialization of `iota_view` (26.6.4.2 [`range.iota.view`]), or

(2.2.4) — a specialization of `subrange` (26.5.4 [`range.subrange`]) where `T::StoreSize` is false,

then `U(ranges::begin(E) + static_cast<D>(F), ranges::end(E))`, except that *E* is evaluated only once, where *U* is `span<typename T::element_type>` if *T* is a specialization of `span` and *T* otherwise.

(2.3) — Otherwise, if *T* is a specialization of `subrange` (26.5.4 [`range.subrange`]) that models `random_access_range` and `sized_range`, then `T(ranges::begin(E) + static_cast<D>(F), ranges::end(E), to-unsigned-like(ranges::distance(E) - static_cast<D>(F)))`, except that *E* and *F* are each evaluated only once.

(2.4) — Otherwise, if *T* is a specialization of `repeat_view` (26.6.5.2 [`range.repeat.view`]):

(2.4.1) — if *T* models `sized_range`, then

`views::repeat(*E.value_, ranges::distance(E) - static_cast<D>(F))`

except that *E* is evaluated only once;

(2.4.2) — otherwise, `((void)F, decay-copy(E))`, except that the evaluations of *E* and *F* are indeterminately sequenced.

(2.5) — Otherwise, `drop_exactly_view(E, F)`.

-3- [Example 1:

```

vector<int> v{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
for (int i : is | views::drop_exactly(6) | views::take_exactly(3)) {
    cout << i << ' '; // prints 6 7 8
}

```

— end example]

[26.7.2.2] Class template `drop_exactly_view` [`range.drop.exactly.view`]

```

namespace std::ranges {
    template<view V>
    class drop_exactly_view : public view_interface<drop_exactly_view<V>> {
    public:
        drop_exactly_view() requires default_initializable<V> = default;
        constexpr explicit drop_exactly_view(V base, range_difference_t<V> count);

```

```

constexpr V base() const & requires copy_constructible<V> { return base_; }
constexpr V base() && { return std::move(base_); }

constexpr auto begin()
    requires (!simple-view<V> && random_access_range<const V>);
constexpr auto begin() const requires random_access_range<const V>;

constexpr auto end() requires (!simple-view<V>)
{ return ranges::end(base_); }

constexpr auto end() const requires range<const V>
{ return ranges::end(base_); }

constexpr auto size() requires sized_range<V> {
    return ranges::size(base_) - static_cast<range_size_t<V>>(count_);
}

constexpr auto size() const requires sized_range<const V> {
    return ranges::size(base_) - static_cast<range_size_t<const V>>(count_);
}

private:
    V base_ = V(); // exposition only
    range_difference_t<V> count_ = 0; // exposition only
};

template<class R>
    drop_exactly_view(R&&, range_difference_t<R>)
        -> drop_exactly_view<views::all_t<R>>;
}

```

```
constexpr explicit drop_exactly_view(V base, range_difference_t<V> count);
```

-1- *Preconditions*: `count <= ranges::distance(base)` is true.

-2- *Effects*: Initializes `base_` with `std::move(base)` and `count_` with `count`.

```
constexpr auto begin() requires (!simple-view<V> && random_access_range<const V>);
constexpr auto begin() const requires random_access_range<const V>
```

-3- *Returns*: `ranges::next(ranges::begin(base_), count_)`.

-4- *Remarks*: In order to provide the amortized constant-time complexity required by the `range` concept when `drop_exactly_view` models `forward_range`, the first overload caches the result within the `drop_exactly_view` for use on subsequent calls.

[*Note 1*: Without this, applying a `reverse_view` over a `drop_exactly_view` would have quadratic iteration complexity. — *end note*]

References

[P2760R1]

Barry Revzin. A Plan for C++26 Ranges. URL: <https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2023/p2760r1.html>